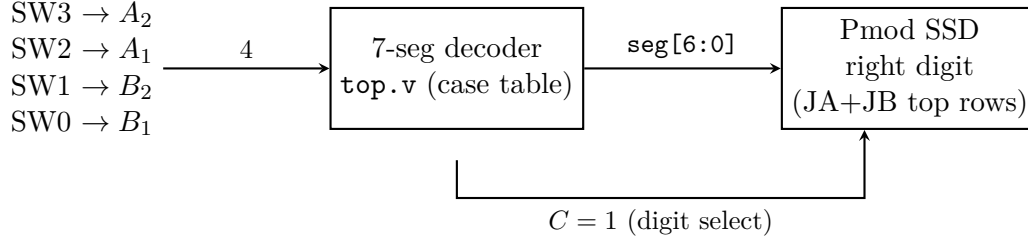


final_8 — Gate-Level Redesign Workspace

NAND / NOR / Inverter implementation of the 7-segment decoder

K-maps are pre-filled from your existing `top.v` truth table. The minimization, groupings, and gate network are yours to derive.

1 Current design at a glance



Signal-to-glyph mapping (bakes in the display's 180° mounting, per `arty.xdc`): `seg[0]` = D (bottom), `seg[1]` = C (lower-right), `seg[2]` = E (lower-left), `seg[3]` = G (middle), `seg[4]` = B (upper-right), `seg[5]` = F (upper-left), `seg[6]` = A (top).

2 K-maps (from your truth table — circle your own groupings)

Each map is one output bit of your `case` statement as a function of $\{A_2, A_1, B_2, B_1\}$. Rows/columns are Gray-coded. Minimize each (SOP or POS), then convert with DeMorgan (Section 3).

seg[0] — viewed D (bottom)

$A_2A_1 \setminus B_2B_1$	00	01	11	10
00	1	1	1	1
01	1	0	1	1
11	1	1	1	1
10	1	1	1	0

seg[1] — viewed C (lower-right)

$A_2A_1 \setminus B_2B_1$	00	01	11	10
00	1	1	1	1
01	1	1	1	0
11	1	1	1	1
10	1	0	1	1

seg[2] — viewed E (lower-left)

$A_2A_1 \setminus B_2B_1$	00	01	11	10
00	1	1	1	1
01	1	0	0	1
11	1	0	0	1
10	1	1	1	0

seg[3] — viewed G (middle)

$A_2A_1 \setminus B_2B_1$	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	1	1	1
10	0	1	1	1

seg[4] — viewed B (upper-right)

$A_2A_1 \setminus B_2B_1$	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	0
10	1	1	0	1

seg[5] — viewed F (upper-left)

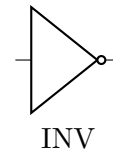
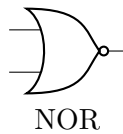
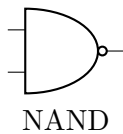
$A_2A_1 \setminus B_2B_1$	00	01	11	10
00	1	1	1	1
01	1	0	0	0
11	1	0	1	1
10	1	0	1	1

seg[6] — viewed A (top)				
$A_2A_1 \backslash B_2B_1$	00	01	11	10
00	1	1	1	1
01	1	0	1	1
11	1	1	1	1
10	1	1	1	0

Tip: several maps are mostly 1s — consider grouping the 0s and building the complement (POS $\rightarrow$ NOR-NOR), then compare gate counts against SOP $\rightarrow$ NAND-NAND.

3 Gate symbols and DeMorgan conversion patterns

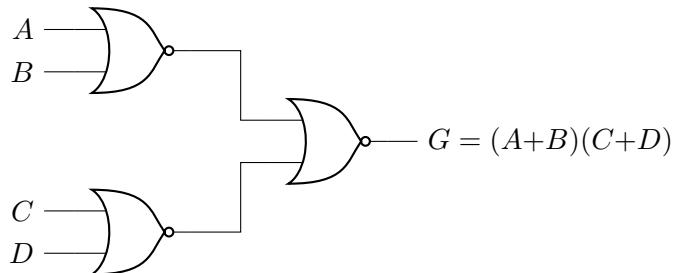
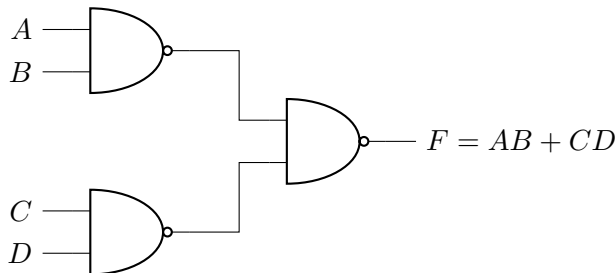
Allowed gates



Textbook identities (generic — not your solution)

Two-level SOP maps to NAND-NAND; two-level POS maps to NOR-NOR:

$$AB + CD = \overline{\overline{AB} \cdot \overline{CD}} \quad (A+B)(C+D) = \overline{\overline{A+B} + \overline{C+D}}$$



Complemented input literals come from inverters on A_2, A_1, B_2, B_1 (at most four inverters, shared across all seven outputs).

Structural Verilog primitives

Output comes first; `nand`/`nor` take any number of inputs.

```

wire nA2, nA1, nB2, nB1;
not  (nA2, A2);           // inverter
nand (w1, A2, nA1, B1);    // 3-input NAND
nand (seg0_x, w1, w2, w3); // final NAND of a NAND-NAND pair
nor  (seg3_x, w4, w5);     // NOR

```

When your gate-level `top.v` is written, run the testbench first to prove equivalence with the case-statement version, then build and flash.